Exercises-2 Testing

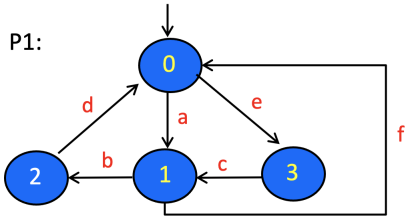
1 McCabe-cyclomatic-based Testing

1. Let $C = \{c_1, c_2, c_3\}$ be a set of linearly independent circuits in a CFG. What does it mean for these circuits to be linearly independent from each other?

   Answer: Imagine that every edge in a graph is given a unique id. A circuit can be represented by a vector over the edge ids, with value 1 if the edge is present in the circuit, and 0 otherwise.

   In a set C of circuits, the circuits are linearly independent from each other if any circuit $c \in C$ cannot be expressed as a linear sum of the remaining circuits in $C/\{c\}$. So, in our example $C = \{c_1, c_2, c_3\}$, c_1 should be linearly independent from $\{c_2, c_3\}$, meaning there is no α, β such that $c_1 = \alpha c_2 + \beta c_3$. Similarly, c_2 should be linearly independent from $\{c_1, c_3\}$; and c_3 should be linearly independent from $\{c_1, c_2\}$.

2. Consider the CFG P_1 shown below, with 0 as the entry node, and 1 as the exit node. The edges have been labelled with ids: $a...e$.



   (a) Consider the circuit $[0, 1, 2, 0]$. How to represent this circuit as a vector over the edges of the graph?

   Answer: The circuit [0,1,2,0] can be represented by the vector $\langle 1, 1, 0, 1, 0, 0 \rangle$. The positions correspond to the edges $\langle a, b, c, d, e, f \rangle$.

   (b) Give five other circuits in P_1. Give few example of circuits which are linearly independent from each other, and few examples which are not.

   Answer: Other circuits in the graph are [0,1,0], [0,3,1,0], [0,3,1,2,0], [0,1,0,3,1,2,0], and [0,3,1,0,1,2,0]. The table below shows their vector representations:

circuits	a	b	c	d	e	f
$c_1 = 0120$	1	1	0	1	0	0
$c_2 = 010$	1	0	0	0	0	1
$c_3 = 0310$	1	0	1	0	1	1
$c_4 = 03120$	0	1	1	1	1	0
$c_5 = 0103120$	1	1	1	1	1	1
$c_6 = 0310120$	1	1	1	1	1	1

   Below are some examples of which of them are linearly independent, and which are not:

   - For example c_5 and c_6 are **not** linearly independent from each other (well... they have the same vector).
   - c_2 and c_4 are linearly independent from each other. One has an edge that the other does not have, implying linear independence.
   - But c_5 is **not** linearly independent from $\{c_2, c_4\}$, as c_5 can be expressed as $c_2 + c_4$.
   - c_4 is also **not** linearly independent from $\{c_1, c_2, c_3\}$ as $c_4 = c_1 + c_3 - c_2$.
   - Every circuit in $C = \{c_1, c_2, c_3\}$ is linearly independent from the other two.

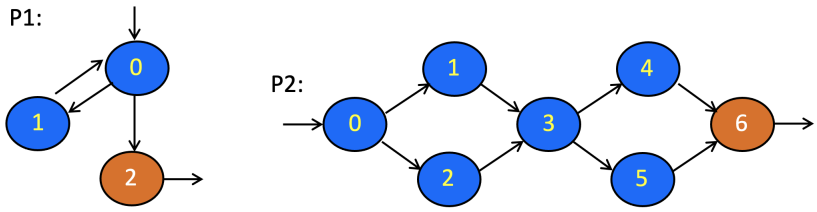
   (c) What is the McCabe cyclomatic number of P_1? Which circuits of P_1 should be included in your test requirement (TR) according to McCabe?

   Answer: The CFG P_1 is already strongly connected so we don't need to add a fake back-edge to the initial node. For McCabe number we can use the formula $E - N + p$ with p=1 (the number of

strongly connected component, which is 1, the CFG itself). So we get McCabe = 6-4+1=3. This gives the maximum number of circuits in the CFG that are linearly independent from each other. For example we can take $C = \{c_1, c_3, c_3\}$ as the TR.

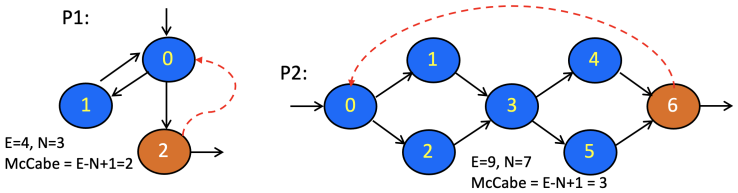
Note: taking just the singleton $C = \{c_5\}$ as the TR might look good as it includes all edges and is certainly linearly independent (because there is only one circuit in C). But its size is not optimal (just one rather than 3, your McCabe number).

3. Consider programs P_1 and P_2 with the CFGs shown below. Node 0 is the entry node. Orange node is the exit node.



(a) What are the McCabe cyclomatic numbers of those programs?

Answer: Both programs are not strongly connected, so we first add a fake back-edge to the initial node as shown below (red edge). On a strongly connected graph, McCabe number is $E - N + 1$. So, this is 2 for P_1 and 3 for P_2.



(b) Give a set C of linearly independent 'circuits' for each of the program, after adding a fake back-edge from its exit node to go back to the entry node. Try to find a C that is *maximal* (containing as many circuits as possible).

Answer: The size of a max-sized C is your McCabe number. For P_1 we have $C = \{010, 020\}$.

For P_2 we have several options. We can take e.g. $C = \{c_1 = 013460, c_2 = 023460, c_3 = 013560\}$.

Note: we don't include the circuit $c_4 = 023560$ in the C for P_2, as c_4 can be expressed in terms of $\{c_1, c_2, c_3\}$, namely $c_4 = c_2 + c_3 - c_1$.

Notice that in both cases (P_1 and P_2) $\#C$ = McCabe number.

(c) Which paths should be in the test requirement (TR) of P_1 and P_2, according to McCabe

Answer: In terms of terms of the modified graphs (one with the fake back edge), the McCabe TR is your C in (b). In terms of the original graphs P_1 and P_2 we remove the fake edge, so we obtain these TRs:

For P_1: originally $\{010, 020\}$, after removing the fake edge we have TR = $\{010, 02\}$.

For P_2: originally $\{013460, 023460, 013560\}$, after removing the fake edge we have TR = $\{01346, 02346, 01356\}$.

(d) Give a minimum sized test-set that gives full McCabe coverage for each program (covering all paths in the TR in your answer in (c)).

Answer: For P_1 we can go with a single test: $t = 0102$.

For P_2 we can go with the test-set T = TR itself (so, consisting of three tests).

2 EFSM

1. Consider a simplified course management system. The system manages a single course. Students can register to the course through the system. Actions possible on the system:

- *add*(*student*) to add a student to the course. This is only possible if the number of students already enrolled to the course is below Max. Else the student is added to the course's waiting list.
- *remove*(*student*) removes a student from the course and its waiting list. If the removed student was not in the waiting list, a student in the waiting list will be enrolled into the course, if the waiting list was not empty.

Initially, the course is empty (no student is enrolled), and its waiting list is of course also empty.

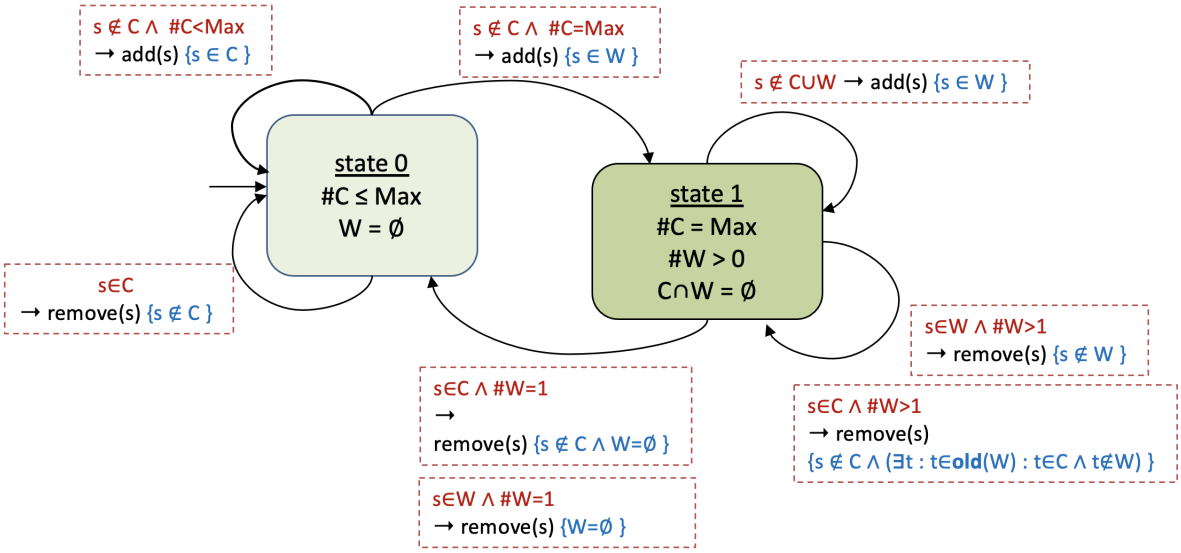
Your tasks:

(a) Propose an EFSM (extended finite state machine) model for the above system. For testing purposes, you can assume that you can inspect variables C and W representing, respectively, the set of students that are currently enrolled to course, and the set of students currently in the waiting list.

Answer: Below is a proposal. State 0 is the initial state. Each transition is labelled by the action that triggers the transition. An arrow with multiple labels (e.g from state 1 back to state 0) represents multiple transitions, each with a single action.

The actions that label the transitions have the form $g \rightarrow action\{Q\}$. The corresponding transition can only be taken when the guard g is true. Q is the action's post-condition; it is expected to hold after the action is taken.

A state is labelled by a number of predicates. Each is expected to hold when the system is in that state.



The post-conditions in the above model are partial. For example, $add(s)$ from state 0 back to state 0 only asserts $s \in C$ as the post-condition. To be more complete you can use $C = \mathbf{old}(C)/\{s\}$ as the post-condition. This would spot a bug where add removes a student from the course.

(b) Give *positive* tests that would cover all prime paths in your model. What should be tested?

Answer: For testing, we'll configure the system to have $Max = 1$. In the model above we only have cyclic prime paths:

- Two $[1, 1]$ cycles, one with an $add(s)$ action and the other with $remove(s)$.
- One $[2, 2]$ cycle labeled with $add(s)$.
- Two $[2, 2]$ cycles labelled with different cases of the $remove(s)$ action.
- Two $[0, 1, 0]$ cycles, using different cases of the $remove(s)$ action.
- Two $[1, 0, 1]$ cycles, also using different cases of the $remove(s)$ action.

A single test can cover all the prime paths in the model above, but let's give a test set of five tests $\{tc_1, ..., tc_5\}$, which are easier to understand. Below, s_0, s_1, s_2 are distinct students.

$tc_1 = add(s_0); remove(s_0)$ (covers both variants 00 cycles)

$tc_2 = add(s_0); add(s_1); add(s_2); remove(s_2)$ (cover two of the three 11 cycles)

$tc_3 = add(s_0); add(s_1); add(s_2); remove(s_0)$ (cover the third 11 cycle)

$tc_4 = add(s_0); add(s_1); remove(s_1); add(s_1)$ (cover one of the two 010 cycles and one of the two 101)

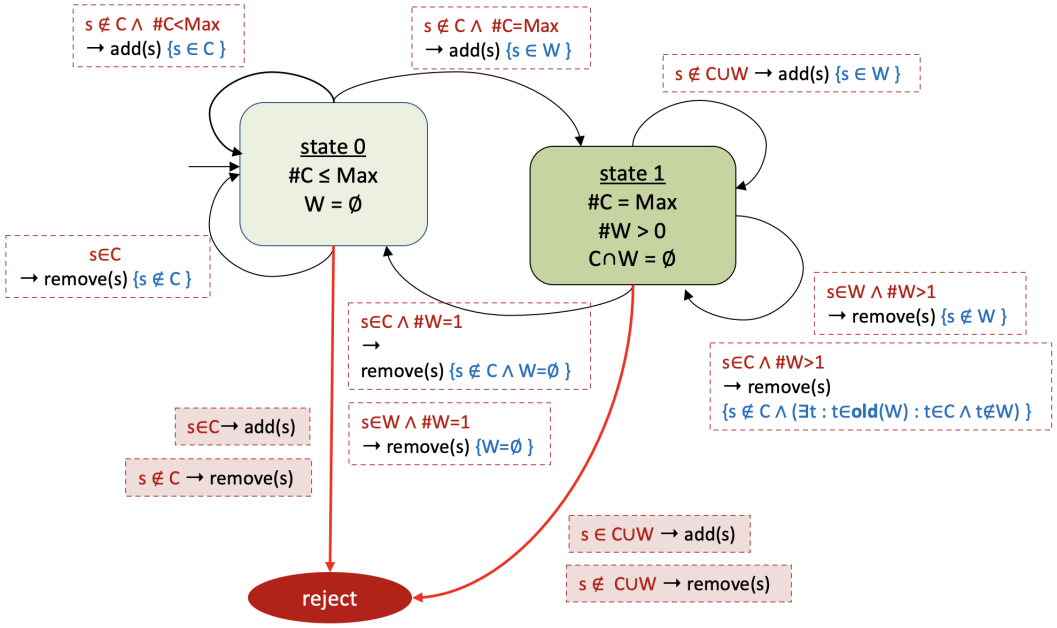
$tc_5 = add(s_0); add(s_1); remove(s_0); add(s_0)$ (cover the other 010 cycle and the other 101 cycle)

What should be tested?

- In every state s (state 0, 1, etc), we have to check that all predicates of s (e.g. $\#C \leq Max \wedge W = \varnothing$ for state 0) indeed hold in that state.
- After every transition with action a, we have to check that a's post-condition holds.

(c) Give *negative* tests that would cover cover all transitions in the reject-state-extended model. What should be tested in these tests?

Answer: Below we show the model after extending it with a *reject* state (red). From every state s in the original model, we add transitions from s to the *reject* state, which should not be possible/allowed in the original model (red transitions). For example, the transition with $add(s)$ from state-0, where s is already in the course ($s \in C$).



A negative test is a sequence of transitions in the modified model, starting at the initial state, and ends in the *reject* state. Such a test expects that the last transition in the sequence should be rejected by the system under test.

We can go with the following six negative tests. The red colored action in every test is supposed to be rejected by the system under test (and else it has a bug).

$tc_1 = add(s_0); add(s_0)$

$tc_2 = add(s_0); rem(s_0); rem(s_0)$

$tc_3 = add(s_0); add(s_1); add(s_2); add(s_1)$

$tc_4 = add(s_0); add(s_1); add(s_2); rem(s_2); rem(s_2)$

$tc_5 = add(s_0); add(s_1); rem(s_0); add(s_1)$

$tc_6 = add(s_0); add(s_1); rem(s_1); rem(s_1)$

2. Imagine an ATM system.

Let's consider the following simplified version of such a system. The user can interact with it using the following actions:

- $insert(C)$: for the user to insert his/her bank-card C to the ATM.
- $pin(n)$: for the user to enter a pincode n for his/her card.
- $withdraw(amount)$: for the user to withdraw cash from the ATM. Only possible if the user has enough in its card balance and the ATM has enough cash. Only a non-negative n can physically be entered, so you don't necessarily need to check that.

- *done*() : to indicate that the user has finished using the ATM.

The following actions are actions of the ATM:

- *eject*() : to eject the card that is currently in the ATM.
- *givecash*(m) : to give the user cash of amount m.

For testing purposes, let's assume the following variables are observable to inspect the ATM's state:

- *cash* : the amount of cash the ATM has left.
- *card* : the card that is currently in the ATM. Null is there is none.
- *balance* : the card's current balance (amount of money the card owner has left in his/her card/bank).
- Additionally, you have access to the predicate $valid(n, C)$ that gives true if n is a valid pincode for the card C, and else false.

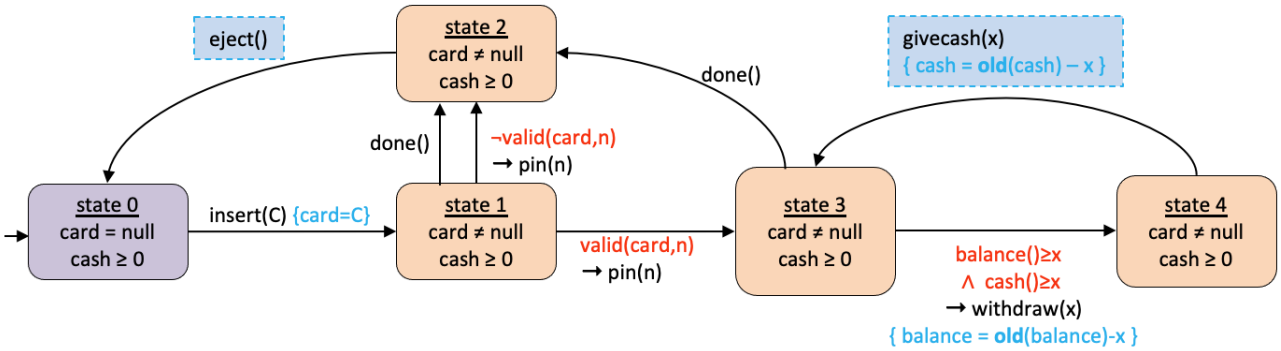
Your tasks:

(a) Propose an EFSM model for the above ATM system.

Answer: Below is a proposal. State 0 is the initial state. Each transition is labelled by the action that triggers the transition. Actions in shaded blue boxes are ATM's output actions. A tester can control input actions, but not output actions.

The action that labels a transition has the general form of $g \rightarrow action\{Q\}$. The corresponding transition can only be taken when the guard g is true. Q is the action's post-condition; it is expected to hold after the action is taken.

A state is labelled by a number of predicates. Each is expected to hold when the system is in that state.



(b) Give a set of *positive* tests with full prime path coverage on your model. What should be tested?

Answer: As in the previous model (Course), the above model for ATM only has cyclic prime paths. We have:

- $[0, 1, 2, 0]$ and its five other variations (so, in total six).
- $[0, 1, 3, 2, 0]$ and its other three variations (in total four).
- $[3, 4, 3]$ and its variation $[4, 3, 4]$.

We can do these tests:

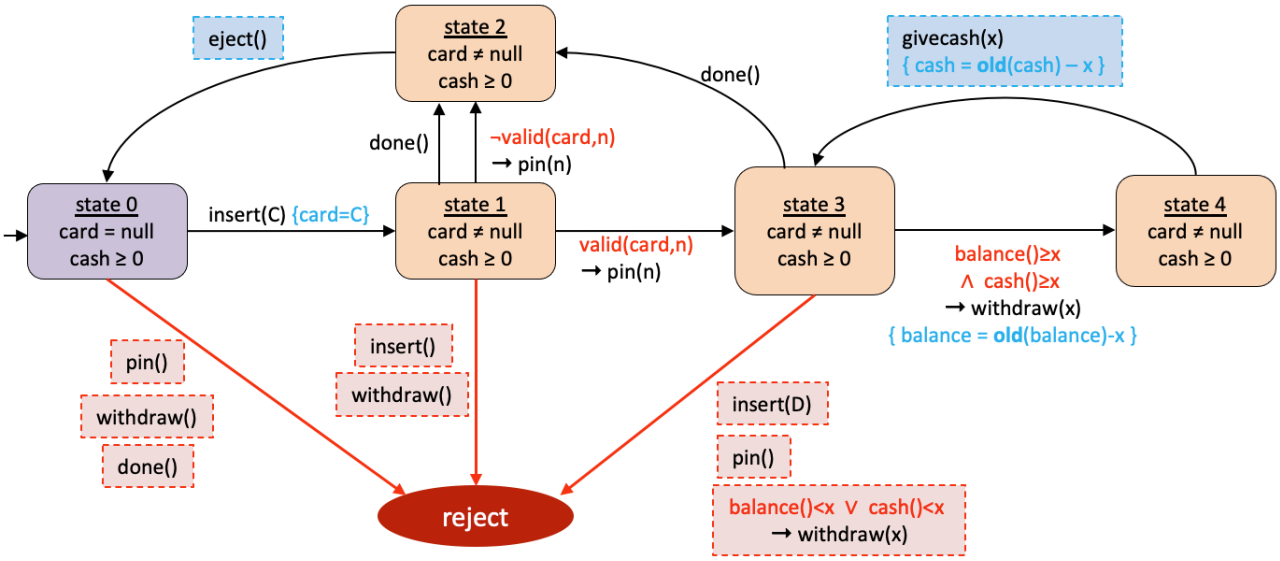
- The tests below will cover the cyclic prime path $[0, 1, 2, 0]$ and all its five other variations. Blue marked actions are output actions, so you should check if the ATM indeed gives the specified output action.
  $tc_1 = [insert(C); done; eject]^2$
  For a sequence of actions z, z^k means repeating z k times.
  $tc_2 = [insert(C); pin(x); eject]^2$, where x is an invalid pincode for the card C.
- The tests below will cover the cyclic prime path $[0, 1, 3, 2, 0]$ and all its variations. n below is a valid pincode for C,
  $tc_3 = [insert(C); pin(n); done; eject]^2$
- The test below covers $[3, 4, 3]$ and its variation $[4, 3, 4]$. The ATM should be set up to initially has at least $x + y$ cash, and a card with balance at least $x + y$ is to be used.
  $tc_4 = insert(C); pin(n); withdraw(x); givecash(x); withdraw(y); givecash(x)$

What should be tested? Because ATM has output actions, it is a bit more involved than the Course management system in question No 1.

- In every state s (state 0, 1, etc), we have to check that all predicates of s (e.g. $\#C \leq Max \wedge W = \varnothing$ for state 0) indeed hold in that state.
- After every transition with action a, we have to check that a's post-condition holds.
- Check that the ATM output actions match the output actions specified in the tests (marked blue in the above tests).

(c) Give a set of *negative* tests that cover all prime paths that in the reject-state extended model, that include the transition $4 \rightarrow 3$ (the ATM gives cash) in the original model. What should be tested in these tests?

Answer: Below is the model we had in (a), extended with the reject state. From every state s in the original model, we add transitions from s to the *reject* state, which should not be possible/allowed in the original model (red transitions). For example, in the state 0, in the original model we can only do inserting a card, and no other actions such as entering pincode, or withdrawing cash. To reflect this, these illegal transitions are now added as transitions towards the reject state.



A negative test is a sequence of transitions in the modified model, starting at the initial state, and ends in the *reject* state. Such a test expects that the last transition in the sequence should be rejected by the system under test.

Now, the prime paths (in the modified model) that pass through the cash withdraw:

- cycles $[3, 4, 3]$ and $[4, 3, 4]$.
- $[4, 3, reject]$
- $[4, 3, 2, 0, reject]$ and $[4, 3, 2, 0, 1, reject]$

To cover those paths, we can go with the following negative tests. The red colored action in every test is supposed to be rejected by the system under test (and else it has a bug).

- The tests below cover $[3, 4, 3]$, $[4, 3, 4]$, and $[4, 3, reject]$. Use an ATM and a card C with initial cash and balance $\geq 2x$ but smaller than $3x$.
  $insert(C); pin(n); wd(x); give(x); wd(x); give(x)$ followed by one of $insert(D)$, $pin()$, or $w(x)$.
- The tests below cover $[4, 3, 2, 0, reject]$.
  $insert(C); pin(n); wd(x); give(x); done; eject$ followed by one of $wd()$, $pin()$, or $done$.
- he tests below cover $[4, 3, 2, 0, 1, reject]$.
  $insert(C); pin(n); wd(x); give(x); done; eject; insert(C)$ followed by either $insert()$ or wd.